

Experiment No. - 3

Student Name: Navjot Kaur

UID: 25MCA20078

Branch: MCA

Section/Group: 1/A

Semester: 3rd

Date of Performance: 07-09-2026

Subject Name: Machine Learning

Subject Code: 25CAP-702

Aim/Overview of the practical: To implement Logistic Regression and k-Nearest Neighbors (k-NN) classification algorithms on a suitable dataset and evaluate their performance using Confusion Matrix, Precision, Recall, and F1-Score.

Objective:

1. To implement Logistic Regression.
2. To implement the KNN algorithm.
3. To classify the heart disease data.
4. To evaluate and compare both models.

Apparatus Used:

- Google Colab
- Python
- Chrome Browser

Code for experiment:

1. Import Required Libraries

```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

2. Upload the Dataset

uploaded = files.upload()

Choose Files heart.csv
heart.csv(text/csv) - 38114 bytes, last modified: 25/10/2019 - 100% done
Saving heart.csv to heart.csv

Figure 1: Successful Upload of the Heart Disease Dataset

3. Load the Dataset

data = pd.read_csv("heart.csv")

4. Display first 5 enteries

df.head()

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

Figure 2: Displaying the First Five Records of the Heart Disease Dataset

4. Display Dataset Information

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1025 non-null   int64
1   sex         1025 non-null   int64
2   cp          1025 non-null   int64
3   trestbps    1025 non-null   int64
4   chol        1025 non-null   int64
5   fbs         1025 non-null   int64
6   restecg     1025 non-null   int64
7   thalach     1025 non-null   int64
8   exang       1025 non-null   int64
9   oldpeak     1025 non-null   float64
10  slope       1025 non-null   int64
11  ca          1025 non-null   int64
12  thal        1025 non-null   int64
13  target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

Figure 3: Displaying Information and Data Types of the Heart Disease Dataset

5. Separate Independent and Dependent Variables

```
X = data.drop("target", axis=1)
```

```
y = data["target"]
```

```
print("Independent Variables:")
```

```
print(X.head())
```

```
print("\nDependent Variable:")
```

```
print(y.head())
```

Independent Variables:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	

	ca	thal
0	2	3
1	0	3
2	0	3
3	1	3
4	3	2

Dependent Variable:

0	0
1	0
2	0
3	0
4	0

Name: target, dtype: int64

Figure 4: Displaying Independent and Dependent Variables of the Dataset

6. Split Dataset into Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=39,
```

```
    stratify=y
```

```
)
```

```
print("Training Data Shape:", X_train.shape)
```

```
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (820, 13)
```

```
Testing Data Shape: (205, 13)
```

7. Scale the Data

```
scaler = StandardScaler()
```

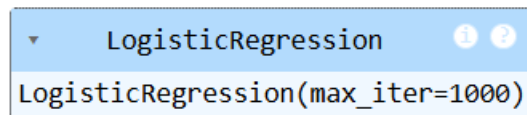
```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

8. Train Logistic Regression Model

```
lr_model = LogisticRegression(max_iter=1000)
```

```
lr_model.fit(X_train_scaled, y_train)
```



```
LogisticRegression  
LogisticRegression(max_iter=1000)
```

Figure 5: Training and Testing Data Shapes

9. Make Predictions

```
y_pred_lr = lr_model.predict(X_test_scaled)
```

10. Display Model Parameters

```
lr_accuracy = accuracy_score(y_test, y_pred_lr)
```

```
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred_lr))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred_lr))
```

Logistic Regression Accuracy: 0.8439024390243902

Confusion Matrix:

```
[[79 21]
 [11 94]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.79	0.83	100
1	0.82	0.90	0.85	105
accuracy			0.84	205
macro avg	0.85	0.84	0.84	205
weighted avg	0.85	0.84	0.84	205

Figure 6: Logistic Regression Model Performance Evaluation

11. Train KNN Model

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
knn_model.fit(X_train_scaled, y_train)
```

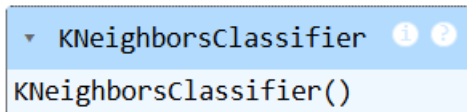


Figure 7: Training the K-Nearest Neighbors (KNN) Model

12. Make KNN Predictions

```
y_pred_knn = knn_model.predict(X_test_scaled)
```

13. Evaluate KNN Model

```
knn_accuracy = accuracy_score(y_test, y_pred_knn)
```

```
print("KNN Accuracy:", knn_accuracy)
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred_knn))
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred_knn))
```

KNN Accuracy: 0.848780487804878

Confusion Matrix:

```
[[85 15]
 [16 89]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.85	0.85	100
1	0.86	0.85	0.85	105
accuracy			0.85	205
macro avg	0.85	0.85	0.85	205
weighted avg	0.85	0.85	0.85	205

Figure 8: K-Nearest Neighbors (KNN) Model Performance Evaluation

14. Compare Both Models

```
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
print("KNN Accuracy:", knn_accuracy)
```

Logistic Regression Accuracy: 0.8439024390243902

KNN Accuracy: 0.848780487804878

Figure 9: Comparison of Logistic Regression and KNN Accuracy

15. Accuracy Comparison Graph

```
models = ["Logistic Regression", "KNN"]
```

```
accuracy = [lr_accuracy, knn_accuracy]
```

```
plt.bar(models, accuracy)
```

```
plt.title("Accuracy Comparison")
```

```
plt.xlabel("Models")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

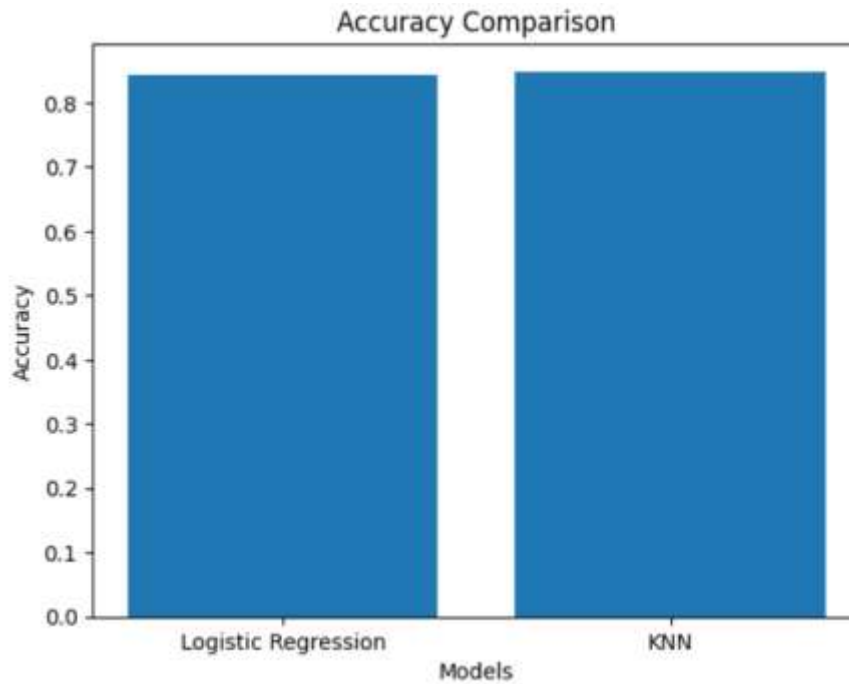


Figure 10: Graphical Comparison of Logistic Regression and KNN Accuracy

Learning Outcomes (What I Have Learnt)

1. Understand Logistic Regression.
2. Understand KNN algorithm.
3. Train classification models.
4. Evaluate model performance.